NEXUS · DOCUMENTATION PACKAGE · DELIVERABLE 01

Product Requirements

Purpose, users, scope, functional requirements, constraints and acceptance criteria for Nexus as implemented.

Document control

Document ID	NX-01-PRD
Version	1.0
Date	2026-11-05
Status	Draft for team review (not approved)
Intended readers	Module assessor reviewing the PoE; developers and testers taking over the codebase
Evidence baseline	Branch New-Changes at commit 3a3f275 (tree identical to main at 9f8e82a)
Prepared by	AI-assisted (Claude Code) from repository evidence; team review pending
Approved by	Unknown No approval recorded

Evidence labels used in this document: Implemented verified in code, configuration or test output; Inferred reasoned from evidence but not executed; Recommendation proposed, not implemented; Open question needs a decision; Unknown no evidence available; Estimate planning figure, not a measurement.

Contents

1. 1 Purpose and product summary
2. 2 Users and roles
3. 3 Scope
    1. 3.1 In scope (implemented)
    2. 3.2 Out of scope or planned (not implemented)
4. 4 Functional requirements
5. 5 Non-functional requirements
6. 6 Constraints and assumptions
7. 7 Success measures
8. 8 Open questions

1 Purpose and product summary

Nexus is an Android app for personal project planning and habit tracking, backed by a small Node.js/Express REST API in `backend/`. The README describes it as "a calm, focused Android workspace for project planning and habit tracking". This document records what the product does at the evidence baseline, what it deliberately does not do, and how each requirement can be accepted. Requirement IDs (`FR-nn`, `NFR-nn`, `CON-nn`) are referenced by every other deliverable.

> **How to read status.** Implemented means the behaviour exists in source at the baseline commit; it does not by itself mean the behaviour was exercised end-to-end on a device for this package. Verified test results are listed in NX-09-TDH. Gaps are tracked as findings in NX-08-AUD and as tasks in NX-06-PLAN.

2 Users and roles

User roles evidenced in code

ID	Role	Description	Evidence
ROLE-01	Visitor (not signed in)	Can view the splash screen, register and sign in. Every other route redirects to Login.	`NexusNavHost.kt` auth guard Implemented
ROLE-02	Signed-in user	Owns their projects, tasks, habits and profile. All API data is partitioned by the JWT `sub` (user id); the Room cache is partitioned by the same owner id.	`server.js` `withAuthAndDb`; `Database.kt` `userId` columns Implemented

There is no administrator, collaborator or shared-project role. The README's test-credential table labels one seeded account "Global system administrator", but the API has no role field or role checks, so that account is an ordinary user Implemented. See AUD-09.

3 Scope

3.1 In scope (implemented)

- Account lifecycle: register, sign in, persisted session, sign out, profile, preferences, password change, account deletion.
- Projects with tasks (due date, priority, status, labels, checklist), search and filters.
- Habits with daily completion, streaks, points, levels, badges and CSV export.
- Offline-first local saves with an ordered upload queue, sync status banner and conflict review.
- English, isiZulu and Setswana for resource-backed strings; light/dark/system theme; animated splash that respects reduced motion.

3.2 Out of scope or planned (not implemented)

From `PROJECT_IMPROVEMENTS.md` ("Still planned") and `docs/POE_PART_2_ADDENDUM.md`: board/calendar/timeline views, recurring tasks and reminders, push notifications, attachments/templates/archive, collaboration/roles/comments, account recovery, Google SSO, general import/export, widgets, complete feature-screen localisation, tablet layouts, and SQLite on the server. None of these exist in code Implemented. A "Task due-date reminders" switch exists in Settings but only stores a preference; see FR-05 and AUD-10.

4 Functional requirements

Acceptance criteria are written so a tester can run them against the debug build and a local API (setup in NX-11-DEP).

Functional requirements and acceptance criteria

ID	Requirement	Acceptance criteria	Evidence / status
FR-01	Register an account with display name, email and password.	Blank fields, a malformed email or a password under 6 characters are rejected on the device before sending. API returns 201 and a JWT; a duplicate email returns 409 `EMAIL_ALREADY_REGISTERED`, which the app should explain to the user.	`AuthViewModel.kt`, `registerSchema` **Implemented**; the app shows a generic message for 409, see AUD-18
FR-02	Sign in and keep the session across app restarts.	Valid credentials return a JWT (24 h expiry) stored in EncryptedSharedPreferences; relaunching the app opens the Dashboard. Wrong credentials (401 `INVALID_CREDENTIALS`) show an error that says the credentials are wrong. Sign-in requires the API; there is no offline sign-in.	`TokenManager.kt`, `tokenFor()` **Implemented**; the current message is "Your session has expired", see AUD-18
FR-03	Sign out.	A confirmation dialog appears; confirming clears the token and returns to Login. Unsent changes stay on the device and upload only when the same account signs in again.	`LogoutConfirmationDialog.kt`, `docs/ONLINE_OFFLINE.md` **Implemented**
FR-04	View and edit the profile display name.	Profile shows display name and email; Settings saves a new display name via `PATCH /users/me`. A cached profile is shown offline.	`ProfileScreen.kt`, `UserRepository.kt` **Implemented**
FR-05	Account preferences: language and notification preference.	Changing language (en/zu/tn) or the reminders switch sends a partial `PATCH /users/me`. If the language update fails it is marked pending and retried. The notifications preference has no delivery mechanism.	`SettingsViewModel.kt` **Implemented**; reminders not delivered, see AUD-10
FR-06	Choose System, Light or Dark theme.	Selection applies immediately at the activity root and persists on the device only (not synced).	`SettingsPreferences.kt`, `MainActivity.kt` **Implemented**
FR-07	Change password.	Requires the current password and a new password of 6 or more characters. Wrong current password returns 401 `INCORRECT_CURRENT_PASSWORD`; success returns 204. Requires the API.	`PasswordDialog.kt`, `/users/me/password` **Implemented**
FR-08	Delete account and all data.	A confirmation dialog ("This permanently deletes your account and synced data.") precedes deletion. The API removes the user, projects, tasks and habits; the app purges that account's local rows and queue, then signs out.	`SettingsScreen.kt`, `DELETE /users/me`, `NexusApp.kt` purge callback **Implemented**
FR-09	Dashboard ("Today") overview.	Shows counts for projects, tasks and activity (completed tasks), plus "Your schedule" cards: Overdue, Due today and the next 3 Upcoming incomplete tasks. Shows "Showing saved data" when served from cache. Refresh also downloads every project's tasks for offline use.	`DashboardScreen.kt`, `DashboardRepository.kt` **Implemented**

ID	Requirement	Acceptance criteria	Evidence / status
FR-10	Create, view, edit, delete and search projects.	A project needs a non-empty name; description is optional. Search filters the list by name. Deleting a project also deletes its tasks locally and on the server.	`ProjectListScreen.kt`, `ProjectDetailScreen.kt`, `ProjectRepository.kt` **Implemented**
FR-11	Create, edit, complete and delete tasks within a project.	A task has title (required), description, due date (yyyy-MM-dd), priority (None/Low/Medium/High), status (To do/In progress/Done), labels and checklist steps. Marking Done sets `isCompleted`; the API keeps `status` and `isCompleted` consistent.	`TaskEditorDialog.kt`, `taskSchema` **Implemented**
FR-12	Find tasks quickly.	Task list offers text search over title and labels, status chips (All, Open, In progress, Done) and a High priority chip; shows progress "x of y tasks complete". A task detail screen shows priority, status, labels and checklist with an Edit action.	`TaskListScreen.kt`, `TaskDetailScreen.kt` **Implemented**
FR-13	Create, edit, delete and complete habits.	A habit has a name and frequency (Daily, Weekly, Custom). "Complete today" toggles today's date in `completedDates`; the button reads "Completed today" when done.	`HabitsScreen.kt`, `HabitCard.kt`, `HabitsViewModel.kt` **Implemented**
FR-14	Habit progress and rewards.	Habits screen shows level, points, current and best streak, completion rate, badges (3-day streak, 7-day streak, Reliable habit builder) and reward suggestions, all computed on the device.	`HabitScoring.kt`, `HabitScoringTest.kt` **Implemented**; accuracy issue AUD-17
FR-15	Export habit history as CSV.	Settings, then "Export habits CSV" builds `habit,frequency,completed_date` rows; "Share CSV" opens the Android share sheet with `text/csv`.	`SettingsViewModel.exportCsv()` **Implemented**; escaping issue AUD-11
FR-16	Work offline and sync later.	Project, task and habit creates, edits and deletes are saved to Room together with an upload record in one transaction, and they survive an app restart. Uploads run on refresh, when the network returns, when you tap Sync, and every 30 s while the process is alive. The banner shows the pending count and state.	`SyncManager.kt`, `NexusApp.kt`, `OfflinePersistenceTest.kt` **Implemented**
FR-17	Handle sync conflicts without losing data silently.	Retried operations are never duplicated (server keeps operation receipts). An edit to an item deleted elsewhere returns 409 and stops the queue; Review offers "Keep changes" or "Discard item edits". Existing-item edits are last-upload-wins.	`/api/sync`, `sync.test.js` **Implemented**
FR-18	Use the app in English, isiZulu or Setswana.	Selecting a language switches the resource-backed strings: auth screens, language picker, profile/settings actions and the stale-data banner (16–17 strings per locale). Most feature-screen text is still hard-coded English.	`values-zu`, `values-tn` **Implemented**; partial, see AUD-09
FR-19	Animated splash and motion that respect accessibility settings.	With system animations disabled, the splash and screen transitions render without animation.	`SplashIntroScreen.kt`, `Motion.kt`, `SplashIntroScreenTest.kt` **Implemented**

ID	Requirement	Acceptance criteria	Evidence / status
FR-20	Show server notices.	The Dashboard shows a banner for maintenance mode or the first announcement from `GET /app/config`. The API always returns maintenance off and no announcements.	`RemoteConfigViewModel.kt`, `/api/app/config` Implemented

5 Non-functional requirements

Quality requirements evidenced in the implementation

ID	Requirement	Evidence / status
NFR-01	Credentials and tokens are protected: bcrypt (cost 12) password hashes; JWT secret of at least 32 characters enforced at startup; token stored encrypted on the device; auth endpoints rate-limited to 10 requests per 15 min per IP.	`server.js`, `TokenManager.kt`, `server.test.js` Implemented
NFR-02	Release builds use HTTPS only; cleartext HTTP is allowed only in debug builds.	`network_security_config.xml` (main/debug), `preReleaseBuild` check Implemented
NFR-03	Data isolation: each account sees only its own server data and cached rows.	Per-user maps on the server; `userId`-scoped DAO queries; sync ownership test Implemented
NFR-04	Durability: a local save is never lost because an upload failed; server writes are atomic (temp file + rename) and serialised by a mutex.	`SyncManager.save()`, `saveDatabase()` Implemented; corrupt-file risk AUD-01
NFR-05	Accessibility: icon buttons carry content descriptions; reduced motion is respected; fallback brand colours were chosen for contrast.	Code review only Implemented; no formal accessibility audit has been run Unknown
NFR-06	Compatibility: Android 7.0 (API 24) and later; target/compile SDK 37.	`app/build.gradle.kts` Implemented
NFR-07	Responsiveness on poor networks: 5 s connect timeout and 15 s call timeout, after which cached data is shown.	`RetrofitClient.kt` Implemented
NFR-08	Observability: structured JSON logs (Pino) with request ids and redaction of passwords and authorization headers; `/health` liveness endpoint.	`server.js` Implemented

6 Constraints and assumptions

Constraints

ID	Constraint	Source
CON-01	The API stores all data in one JSON file and must run as a single instance; replicas must not share the file.	`docs/ONLINE_OFFLINE.md`, `server.js` Implemented
CON-02	No public API host has been provisioned. Release builds cannot be produced without an HTTPS `NEXUS_API_URL`.	README, `app/build.gradle.kts` Implemented
CON-03	Background upload is not guaranteed while the app process is stopped; there is no WorkManager job.	`docs/ONLINE_OFFLINE.md`, `NexusApp.kt` Implemented

ID	Constraint	Source
CON-04	The project has no PostgreSQL or other database server, and the PoE material must not present one.	`docs/POE_PART_2_ADDENDUM.md` Implemented
CON-05	Data that was never downloaded to a device cannot appear offline.	`docs/ONLINE_OFFLINE.md` Implemented

7 Success measures

The repository defines no product metrics or analytics. `PROJECT_IMPROVEMENTS.md` asks that each feature define "one measurable success outcome", but none is recorded Unknown. Recommendation Before the final PoE, agree on two or three measures that can be checked during testing, for example "a task created offline appears on the server within one sync cycle of reconnecting" (FR-16).

8 Open questions

Decisions needed from the team

ID	Question	Related
OQ-01	Should the "Task due-date reminders" switch be hidden until reminders exist, or should reminders be built for the final PoE? Open question	FR-05, AUD-10, IMP-08
OQ-02	Which public host will run the API, and who owns its secret and backups? Open question	CON-02, NX-11-DEP
OQ-03	Is a "Global system administrator" role actually required? If so it needs server-side authorisation; if not, the README label should be corrected. Open question	ROLE-02, AUD-09
OQ-04	Which of the planned final-PoE features (Google SSO, push notifications, full localisation) are committed, and in what order? Open question	NX-06-PLAN